

Class 6: R functions

Erin (PID: A17713992)

Table of contents

Background	1
A first function	1
Q2. A generate_dna() function	3
A generate_protein() function	5
Are our peptides “unique in nature”?	6
Connecting our findings to immunology	7

Background

All functions in R have at least 3 things:

- a *name* (we pick that and use it to call the function)
- input *arguments* (one or more comma separated inputs that go inside the brackets when we call the function)
- the *body* (the lines of R code that do the work of the function)

A first function

Here we will create a function to add some numbers. Let’s call it `add()`

Input arguments can be either “**required**” or “**optional**”. The latter have fall-back default values that will be used if the user does not specify them.

Q1a. Your first version of the function should add two input numbers together. For example, `add(4, 7)` should return 11.

```
add <- function(x, y=0) {  
  x + y  
}
```

Can we use our new function:

```
add(10, 100)
```

```
[1] 110
```

```
add(10)
```

```
[1] 10
```

Q1b. For your second version, adapt your first function so it can take a single input vector or two inputs as before. For example, `add(4, 7)` and `add(c(4,7,10))`.

```
add <- function(x, y=0) {  
  sum(x, y)  
}
```

```
add(c(4, 7, 10))
```

```
[1] 21
```

Q1c. Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, `add(1, 2, 3, -4)` should return 2. Hint: explore the `...` (dots) argument or the base R function `sum()`.

```
add <- function(x, y, ...) {  
  sum(x, y, ...)  
}
```

```
add(1, 2, 3, 4)
```

```
[1] 10
```

We can explicitly set a **return** value output from a function (rather than the default last line) by using the `return()` function call.

```
add <- function(x, y=0, z=0) {
  sum(x, y, z)
  cat("Is it break time yet?\n")
}

add(10, 100)
```

Is it break time yet?

```
add <- function(x, y=0, z=0) {
  return(sum(x, y, z))
  cat("Is it break time yet?\n")
}

add(10, 100)
```

[1] 110

Q2. A generate_dna() function

A useful function here is the “base R” `sample()` function:

```
sample(1:5, size=60, replace=TRUE)
```

```
[1] 1 5 2 5 4 1 4 3 3 1 4 4 2 5 4 4 2 5 5 4 5 1 1 3 3 1 2 4 2 1 3 2 4 2 5 3 2 4
[39] 5 3 3 2 2 3 1 1 5 2 2 5 5 3 3 5 2 3 5 4 4 4
```

We can use this to make a random nucleotide sequence if we draw from “A”, “C”, “G”, and “T”...

```
sample(x=c("A", "C", "G", "T"), size = 10, replace = TRUE)
```

```
[1] "T" "A" "T" "G" "T" "G" "A" "C" "C" "C"
```

Q2a. Write a function `generate_dna()` that returns a random DNA sequence of a length specified by the user. Your first version should return a multi-element vector of single character nucleotides. For example `generate_dna(6)` might return “A”, “T”, “T”, “G”, “A”, “C”.

```
generate_dna <- function(len) {
  sample(x=c("A", "C", "G", "T"), size = len, replace = TRUE)
}
```

```
generate_dna(len = 6)
```

```
[1] "C" "A" "T" "C" "C" "C"
```

Q2b. Your second version should *optionally* be able to return either a multi-element vector of single character nucleotides (as before) or a **single character string** (not a vector of single letters but a single vector of multiple letters). For example “AAGGCTG”.

```
generate_dna <- function(len, single.element = TRUE) {
  ans <- sample(x=c("A", "C", "G", "T"), size = len, replace = TRUE)
  if(single.element) {
    ans <- paste(ans, collapse = "")
  }
  return(ans)
}
```

```
generate_dna(len = 6)
```

```
[1] "CCCCCA"
```

Functions that could be useful here are `paste()`, `if()`, `cat()`, and `return()`.

```
paste( c("A", "C", "G"), collapse = "---")
```

```
[1] "A---C---G"
```

Q2c. Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length.

```
>len9
CGAAGGCTG
```

```

generate_dna <- function(len, single.element = TRUE) {
  ans <- sample(x=c("A", "C", "G", "T"), size = len, replace = TRUE)
  if(single.element) {
    ans <- paste(ans, collapse = "")
  }

  ## Format as FASTA with an ID line
  cat( paste(">len", len, "\n", sep = "") )
  cat(ans)
  cat("\n")
  ##

  return(ans)
}

```

```
x <- generate_dna(4)
```

```

>len4
AGCA

```

A generate_protein() function

Q3. Write a function `generate_protein()` that returns a random peptide/protein sequence of a length specified by the user. For example `generate_protein(6)` might return "WQRTAG".

```

generate_protein <- function(len) {
  ## choose amino acids at random according to specified length
  ans <- sample(x=c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F", "P", "T", "W", "Y", "V"), size = len, replace = TRUE)

  ## concatenate the sequence
  ans <- paste(ans, collapse = "")
  return(ans)
}

```

```
generate_protein(len = 6)
```

```
[1] "HTQKLY"
```

Q4. Generate random protein sequences of length 6 to 13. Adapt and use your `generate_protein()` function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function `for()` or `sapply()` so that you do not have to call `generate_protein()` eight times by hand.

```
for(l in 6:13) {  
  cat(">", l, "\n", sep = "")  
  cat( generate_protein(l), "\n" )  
}
```

```
>6  
KRRVCA  
>7  
GPFQHFQ  
>8  
LMDNCTAT  
>9  
PYFFCMTMT  
>10  
MYIAGMQIMS  
>11  
HRCPCATRNYS  
>12  
RDTHDINNPA  
>13  
PSQDMFNSNSSMM
```

Are our peptides “unique in nature”?

Q5. Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database at <https://blast.ncbi.nlm.nih.gov/>. For this question do not restrict the organism (leave the Organism field blank so that the full nr database is searched).

Length (aa)	Best hit % identity	Best hit % coverage	Unique
6	100	100	N
7	100	100	N
8	88	100	Y
9	89	100	Y

Length (aa)	Best hit % identity	Best hit % coverage	Unique
10	90	100	Y
11	100	50	Y
12	92	75	Y
13	77	90	Y

Q5a. At which sequence length do your randomly generated peptides start to look “unique in nature” (i.e. no 100% coverage + 100% identity hit)?

At sequence length 8, my randomly generated peptides start to look “unique in nature”.

Q5b. Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space (20^L for a peptide of length L) and to the size of the known protein universe.

The total possible combinations of amino acids for shorter sequences are much smaller than for longer sequences, as supported by the exponential growth formula 20^L . As a result, there is statistically a higher chance of short peptides appearing in nature by chance, whereas longer peptides would be much harder to find in nature. In addition, the size of the known protein universe is approximately 10^8 – 10^9 , which displays the great number of possible peptide sequences that can exist.

Connecting our findings to immunology

Q6. In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides?

Based on the reasoning from the previous question, the minimum length most likely is around at least 10 amino acids. Presenting shorter peptides would be a bad design choice for the immune system because these lengths would not be unique in nature. This would ultimately result in a high binding affinity for self immune cells, causing the body to attack its own cells. Having longer amino acid lengths ensures stability and the ability to bind to unique cells.